

OOPS(Object Oriented Programming System) in Python

What is oops?

It is a programming concept used to organize code using classes and objects.

Why OOPS is Used

- 1. Code Reusability** -- We can reuse code again and again using inheritance.
- 2. Easy Maintenance** -- Large programs become easier to manage and update.
- 3. Security** -- Data can be protected using encapsulation.
- 4. Real-world Modeling**
 - Car
 - Student
 - Mobile
 - Bank Account
- 5. Reduces Code Duplication** -- Same code does not need to be written many times.

Class

What is class?

- A Class is a blueprint or template used to create objects.
- Class ek blueprint ya template hoti hai jisse objects banaye jate hain.

Class ke andar:

- Variables (Data)
- Methods (Functions)

Class khud real object nahi hoti, balki object banane ka design hoti hai.

Syntax

class **ClassName:**

Variable

Method

Example 1

class Student: # Class banayi

name = "Suraj" # Variable banaya

print(Student.name) # Variable access kiya

Example 2

```
class Car:  
    def start(self):  
        print("Car Started")  
car1 = Car()  
car1.start()
```

Explanation

- **Car → Class**
- **start() → Method**
- **car1 = Car() → Object create hua**
- **car1.start() → Method call hua**

Object

What is Object?

- An **Object** is a real instance of a class.
- Objects are used to access variables and methods of a class.
- Object ki help se hum class ke variables aur methods ko use karte hain.

Syntax

```
object_name = ClassName()
```

Example 1

```
class Student:
```

```
    name = "Suraj"
```

```
obj = Student() # Student → Class name
```

```
print(obj.name) # name → Variable
```

Example 2

```
class Car:
```

```
    def start(self): # Car is a class
```

```
        print("Car Started") # start() is a method
```

Constructor

What is a Constructor?

- A constructor is a special method in Python that is automatically called when an object is created. It is used to initialize the object's data.
- Constructor ek special method hai jo object banate hi automatically execute hota hai aur object ki initial values set karta hai.

Why do we use a Constructor?

- To initialize object data.
- To assign initial values.
- To reduce repeated code.
- To prepare the object for use.

Advantages of Constructor

- Saves time
- Makes code cleaner
- Initializes data automatically
- Improves readability
- Prevents uninitialized objects

What is `__init__()`?

- Special method
- Built-in constructor
- Automatically executed
- Initializes object variables

Role of **self** in the Constructor?

self is a reference to the current object (instance) of the class. It allows methods to access instance variables and other instance methods.

self current object (instance) ka reference hota hai. Iski madad se object ke variables aur methods ko access kiya jata **hai**.

Example:

class Student:

def __init__(self, name):

self.name = name # Stores the value in the current object.

s1 = Student("Suraj")

print(s1.name) #Accesses the value stored in that object.

Types of Constructor?

- **Default (Non-Parameterized) Constructor**

A constructor that has only the **self** parameter and does not accept any additional arguments is called a Default (Non-Parameterized) Constructor.

Syntax

```
class ClassName:  
    def __init__(self):  
        # initialization code
```

Example:

```
class Student:  
    def __init__(self):  
        print("Default Constructor Called")  
  
s1 = Student()
```

2. Parameterized Constructor

A constructor that accepts one or more parameters (in addition to `self`) is called a Parameterized Constructor.

Syntax

```
class ClassName:  
    def __init__(self, parameter1, parameter2):  
        # initialization code
```

Example

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

s1 = Student("Suraj", 20)
print(s1.name)
print(s1.age)
```

Explanation

- "Suraj" is assigned to name.
- 20 is assigned to age.
- self.name and self.age become instance variables of the object.

3. Default Constructor

Default Constructor wo constructor hota hai:

- jisme sirf self parameter hota hai
- extra values pass nahi hoti
- object bante hi automatically call hota hai

Syntax

```
class ClassName:  
    def __init__(self):  
        # code
```

Example

```
class Student:  
    def __init__(self):  
        print("Default Constructor Called")  
s1 = Student()
```

Variable Initialize

```
class Mobile:  
    def __init__(self):  
        self.company = "Samsung"  
        self.price = 25000  
m1 = Mobile()  
  
print(m1.company)  
print(m1.price)
```

Variable

What is Variable

1. Variable is a variable that belongs to a class or an object.
2. It stores the data (properties) of an object.
3. Variable describe the characteristics of an object.

Syntax

```
class Student:  
    def __init__(self, name):  
        self.name = name    # Attribute
```

Example

```
class Student:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
s1 = Student("Suraj", 21)  
  
print(s1.name)  
print(s1.age)
```

Types of Variable

1. Instance Variable

- Instance Variable har object ka **alag (unique) data** store karta hai.
- Ye constructor (`__init__`) ke andar self ke through banta hai.
- Har object ki value alag ho sakti hai.

Syntax

```
class Student:  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

Example

```
class Student:  
  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
s1 = Student("Suraj", 20)  
s2 = Student("Rahul", 22)  
  
print(s1.name)  
print(s1.age)  
print(s2.name)  
print(s2.age)
```

2. Class Variable

- Class Variable poori class ka common data store karta hai.
- Iski sirf ek hi copy hoti hai.
- Sabhi objects ise share karte hain.

Syntax

```
class Student:  
    college = "ABC College"
```

Example

```
class Student:  
    college = "ABC College"  
    def __init__(self, name):  
        self.name = name
```

```
s1 = Student("Suraj")  
s2 = Student("Rahul")
```

```
print(s1.name)  
print(s1.college)  
print(s2.name)  
print(s2.college)
```

- **Easy Example**

- `class Student:`
 `college = "ABC College"      # Class Variable`
 `def __init__(self, name, age):`
 `self.name = name          # Instance Variable`
 `self.age = age            # Instance Variable`
`s1 = Student("Suraj", 20)`
`s2 = Student("Rahul", 22)`

`print(s1.name, s1.age, s1.college)`
`print(s2.name, s2.age, s2.college)`

- **Difference Between Instance Variable and Class Variable**

Instance Variable	Class Variable
<code>self</code> ke through banta hai	Class ke andar directly banta hai
Har object ki alag copy hoti hai	Sirf ek copy hoti hai
Object specific data store karta hai	Common data store karta hai
<code>self.name</code>	<code>college = "ABC"</code>
Har object ki value alag ho sakti hai	Sab objects ki value same hoti hai

Methods?

- A method is a function that is defined inside a class and is used to perform operations on objects or classes.
- Method ek function hota hai jo class ke andar banaya jata hai. Iska use object ya class par koi kaam (operation) karne ke liye hota hai.

Syntax

```
class ClassName:  
    def method_name(self):  
        # code
```

Example

```
class Student:  
    def show(self):  
        print("Hello Student")  
  
s1 = Student()  
s1.show()
```

Types of Methods

- Python me mainly 3 types ke methods hote hain.

1.Normal (Instance) Method

- A normal method works with the object (instance) of the class. It can access instance variables using self.
- Normal Method object ke saath kaam karta hai. Ye object ke data (instance variables) ko access karta hai aur isme self parameter zaroor hota hai.

Syntax

```
class ClassName:  
    def method_name(self):  
        # code
```

Example

```
class Student:  
    def __init__(self, name):  
        self.name = name  
    def display(self):  
        print("Student Name:", self.name)
```

```
s1 = Student("Suraj")  
s1.display()
```

2. Class Method

- A class method works with the **class** instead of the object. It uses **cls** as the first parameter and is decorated with **@classmethod**.
- Class Method object ke bajay poori class ke saath kaam karta hai. Isme pehla parameter **cls** hota hai aur **@classmethod** likhna padta hai.

Syntax

```
class ClassName:  
    @classmethod  
    def method_name(cls):  
        # code
```

Example

```
class Student:  
    school = "ABC School"  
  
    @classmethod  
    def show_school(cls):  
        print("School:", cls.school)  
Student.show_school()
```


3. Static Method

- A static method neither works with the object nor the class. It is an independent utility method and does not use self or cls.
- Static Method na object ke saath directly kaam karta hai aur na class ke saath. Ye sirf ek normal function ki tarah class ke andar hota hai.

Syntax

```
class ClassName:  
    @staticmethod  
    def method_name():  
        # code
```

Example

```
class Student:  
    @staticmethod  
    def greet():  
        print("Welcome Students")  
  
Student.greet()
```

super() Function

What is super()

The super() function is used to:

- access the parent class
- call parent class methods
- call parent class constructors

Why Do We Use super()?

- to reuse parent class code
- to call parent constructor
- to call parent methods
- to avoid writing duplicate code

Basic Syntax

Parent Constructor

```
super().__init__()
```

Parent Method

```
super().method_name()
```

Example

```
class Parent:
    def show(self):
        print("Parent Method")
class Child(Parent):
    def display(self):
        super().show()
c1 = Child()
c1.display()
```

Constructor Example

```
class Parent:
    def __init__(self):
        print("Parent Constructor")
class Child(Parent):
    def __init__(self):
        super().__init__()
        print("Child Constructor")
c1 = Child()
```

Parameterized Constructor Example

```
class Parent:
    def __init__(self, name):
        self.name = name
class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name)
        self.age = age
    def show(self):
        print("Name:", self.name)
        print("Age:", self.age)
c1 = Child("Suraj", 20)
c1.show()
```

Advantages of super()

- Reuses parent code
- Makes code cleaner
- Reduces duplicate code
- Easy parent access

4 Pillars of OOPS

Inheritance

What is Inheritance ?

- Ek class dusri class ke properties aur methods ko use karti hai, ise Inheritance bolte hain.
- Inheritance is an OOPs concept in which one class acquires the properties and methods of another class.

Why We Use Inheritance

- Code Reusability
- Less Code
- Easy Maintenance
- Faster Development

Syntax

```
class Parent:
```

```
# parent properties and methods
```

```
pass
```

```
class Child(Parent):
```

```
# child properties and methods
```

```
pass
```

Example

```
class Father:
    def bike(self):
        print("Father has Bike")
class Son(Father):
    pass
s = Son()
s.bike()
```

Parent Class

The class whose properties and methods are inherited is called the Parent Class.

Example

```
class Animal:
    def sound(self):
        print("Animal makes sound")
```

Child Class

The class that inherits another class is called the Child Class.

Example

```
class Dog(Animal):
    pass
```

Parent Constructor

Child Class Own Method

```
class Father:  
    def house(self):  
        print("Father House")
```

```
class Son(Father):  
    def car(self):  
        print("Son Car")
```

```
s = Son()
```

```
s.house()
```

```
s.car()
```

Types of Inheritance

- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

Single Inheritance

- When one child class inherits one parent class, it is called Single Inheritance.
- Ek child ek parent se inherit karta hai.

Syntax

```
class A:
```

```
    pass
```

```
class B(A):
```

```
    Pass
```

Example

```
class Father:
```

```
    def bike(self):
```

```
        print("Father has a bike")
```

```
class Son(Father):
```

```
    def mobile(self):
```

```
        print("Son has a mobile")
```

```
s = Son()
```

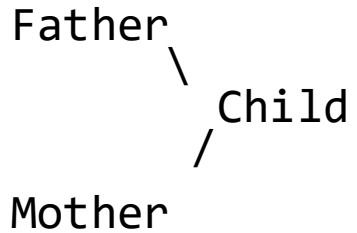
```
s.bike()
```

```
s.mobile()
```


❓ Multiple Inheritance

- jab koi child class ek se zyada parent class ko inherit karti hai, to ise multiple inheritance kaha jata hai.
- When one child class inherits multiple parent classes, it is called Multiple Inheritance.

Diagram



Syntax

Example

```
class Father:
    def bike(self):
        print("Father Bike")
class Mother:
    def jewelry(self):
        print("Mother Jewelry")
class Child(Father, Mother):
    pass
c = Child()
c.bike()
c.jewelry()
```

Multilevel Inheritance

- When a class inherits another inherited class, it is called Multilevel Inheritance.
- jab koi class kisi dusri inherited class ko inherit karti hai, to ise multilevel inheritance kaha jata hai.

Diagram

GrandFather → Father → Son

Example

```
class GrandFather:
    def land(self):
        print("GrandFather Land")

class Father(GrandFather):
    def house(self):
        print("Father House")

class Son(Father):
    pass

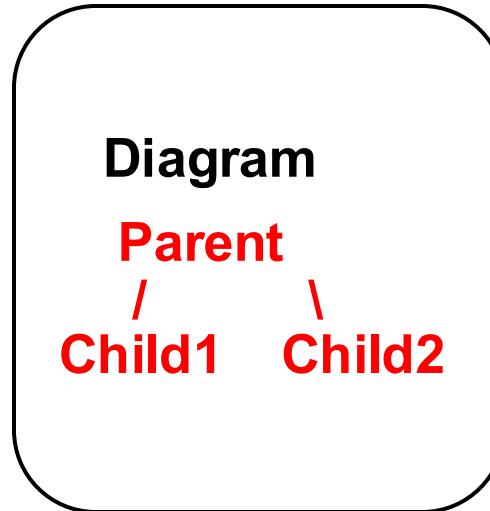
s = Son()
s.land()
s.house()
```

Hierarchical Inheritance

- When multiple child classes inherit one parent class, it is called Hierarchical Inheritance.
- jab kai child class ek hi parent class ko inherit karti han, to ise hirarkikal inheritance kaha jata hai.

Example

```
class Father:  
    def house(self):  
        print("Father House")  
class Son(Father):  
    pass  
class Daughter(Father):  
    pass  
s = Son()  
d = Daughter()  
s.house()  
d.house()
```



Hybrid Inheritance

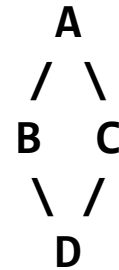
- Combination of two or more inheritance types is called Hybrid Inheritance.
- Jab ek program me do ya do se adhik inheritance types ko combine kiya jata hai, use Hybrid Inheritance kehte hain.

Example

```
class A:
    def showA(self):
        print("Class A")
class B(A):
    def showB(self):
        print("Class B")
class C(A):
    def showC(self):
        print("Class C")
class D(B, C):
    def showD(self):
        print("Class D")

obj = D()
obj.showA()
obj.showB()
obj.showC()
obj.showD()
```

Diagram



Method Overriding

- Method Overriding is an OOP feature in which a child class provides its own implementation of a method that is already defined in the parent class.
- method overwriding mein child clas, parent class ke method ko ek hi naam aur ek hi parameters ke saath framework paribhashit karta hai. jab object child class hota hai, to child class wala method nishpadit hota hai.

Syntax

```
class Parent:
    def method_name(self):
        # Parent Method

class Child(Parent):
    def method_name(self):
        # Overridden Method
```

Example

```
class Animal:  
    def sound(self):  
        print("Animal makes a sound")  
  
class Dog(Animal):  
    def sound(self):  
        print("Dog barks")  
d = Dog()  
d.sound()
```

Important Points

- Child class parent method ko replace karti hai.
- Method name same hona chahiye.
- Parameters bhi same hone chahiye.
- Runtime Polymorphism ka example hai.
- Inheritance ke bina Method Overriding nahi hoti.

isinstance() Function

- `isinstance()` checks whether an object is an instance of a specified class or its parent class.
- `isinstance()` check karta hai ki object kisi class ya uski parent class ka object hai ya nahi.

Syntax

`isinstance(object, ClassName)`

Parameters

- `object` → Object to check
- `ClassName` → Class name

Important Points

- Object ka type check karta hai.
- Parent class ko bhi True return karta hai.

- Return value sirf True ya False hoti hai.
- Type checking ke liye use hota hai.

Example

```
class Animal:  
    pass  
class Dog(Animal):  
    pass  
  
d = Dog()  
print(isinstance(d, Dog))  
print(isinstance(d, Animal))
```


issubclass() Function

- `issubclass()` checks whether one class is a subclass of another class.
- `issubclass()` check karta hai ki ek class dusri class ki child (subclass) hai ya nahi.

Syntax

`issubclass(ChildClass, ParentClass)`

Parameters

- `ChildClass`
- `ParentClass`

Example

```
class Animal:  
    pass  
class Dog(Animal):  
    pass  
print(issubclass(Dog, Animal))  
print(issubclass(Animal, Dog))
```

Important Points

- Classes ke relationship ko check karta hai.
- Object par nahi, sirf classes par use hota hai.
- Return value True ya False hoti hai.
- Inheritance verify karne ke liye use hota hai.

Encapsulation

What is Encapsulation?

- Encapsulation ka matlab hota hai data aur methods ko ek hi class ke andar bandhna (wrap karna).
- Iska use **data ko secure/hide** karne ke liye bhi hota hai.
- Encapsulation is the process of wrapping **data (variables)** and **methods (functions)** into a single unit called a **class**.

Syntax

```
class ClassName:  
    def __init__(self):  
        self.__variable = value    # Private Variable  
  
    def method_name(self):  
        # code
```

Example

```
class Student:  
  
    def __init__(self):  
        self.__name = "Suraj"  
  
    def show(self):  
        print(self.__name)  
  
s1 = Student()  
  
s1.show()
```

What is Data Hiding?

- Data Hiding means restricting direct access to data for security.
- Data Hiding ka matlab data ko direct access se secure/hide karna hota hai.

Syntax

`self.__variable = value`

Example

```
class Bank:
    def __init__(self):
        self.__balance = 5000
b1 = Bank()

print(b1.__balance)
```

What is Private Variable

- A private variable is a variable that cannot be accessed directly outside the class.
- Private variable wo variable hota hai jise class ke bahar directly access nahi kar sakte.

Syntax

```
self.__name = value
```

Example

```
class Student:  
    def __init__(self):  
        self.__name = "Suraj"  
    def show(self):  
        print(self.__name)  
  
s1 = Student()  
s1.show()
```

Private Method

- A Private Method is a method that cannot be accessed directly from outside the class. It is created by adding double underscore (__) before the method name.
- Private Method ek aisa method hota hai jise class ke bahar se directly access nahi kiya ja sakta. Iske naam ke aage double underscore (__) lagaya jata hai.

Syntax

```
class ClassName:  
    def __private_method(self):  
        # Code  
  
    def public_method(self):  
        self.__private_method()
```

Accessing a Private Method

```
class Student:  
    def __show(self):  
        print("This is a Private Method")  
    def display(self):  
        self.__show()  
  
s = Student()  
s.display()
```

Explanation

- `__show()` ek private method hai.
- Isko class ke bahar se direct call nahi kar sakte.
- `display()` (public method) ke through `__show()` ko call kiya gaya hai.

Protected Variable

- Protected Variable ka use class aur uski child class ke liye hota hai. Iske aage ek underscore (_) lagaya jata hai.
- A Protected Variable is intended to be used within the class and its child classes. It is written with a single underscore (_).
- **Note:** Python me protected variable ko bahar se access kiya ja sakta hai, lekin aisa karna recommended nahi hota.

Syntax

```
class Student:  
    def __init__(self):  
        self._name = "Suraj"
```

Example

```
class Student:  
    def __init__(self):  
        self._name = "Suraj"
```

```
s = Student()  
print(s._name)
```

Explanation

- _name ek **protected variable** hai.
- Single underscore (_) sirf ye batata hai ki ise direct access nahi karna chahiye.
- Python ise strictly restrict nahi karta.

Public Variable

- A Public Variable is a variable that can be accessed and modified directly from anywhere, both inside and outside the class.
- Public Variable ek aisa variable hota hai jise class ke andar aur class ke bahar dono jagah se directly access aur modify kiya ja sakta hai.

Why is Public Variable Used?

- Public variables are used to:
- Store object data.
- Allow direct access to data.
- Make code simple and easy.
- Share data without restrictions.

Syntax

```
class ClassName:  
    def __init__(self):  
        self.variable_name = value
```

Example

```
class Student:  
    def __init__(self):  
        # Public Variable  
        self.name = "Suraj"  
s = Student()  
  
print(s.name)
```

Public Variable Example (Access)

```
class Student:  
    def __init__(self):  
        self.name = "Suraj"  
        self.age = 22  
  
s = Student()  
print(s.name)  
print(s.age)
```

Public Variable Example (Modify)

```
class Student:  
    def __init__(self):  
        self.name = "Suraj"  
  
s = Student()  
print(s.name)  
s.name = "Rahul"  
print(s.name)
```

What is Security?

- Security means protecting data from unauthorized access.
- Security ka matlab unauthorized access se data ko protect karna hota hai.

Syntax

```
if condition:  
    self.__variable = value
```

Example

```
class Employee:  
    def __init__(self):  
        self.__salary = 50000  
    def set_salary(self, amount):  
        if amount > 0:  
            self.__salary = amount  
        else:  
            print("Invalid Salary")  
    def get_salary(self):  
        return self.__salary  
e1 = Employee()  
e1.set_salary(-1000)  
print(e1.get_salary())
```

Access through Methods

- Private data is accessed using methods like Getter and Setter.
- Private data ko methods ke through access kiya jata hai.

Syntax

`obj.get_data()`

`obj.set_data()`

What is Getter

- A Getter is a method used to get/read/access the value of a private variable.
- Getter ek method hota hai jo private variable ki value ko access(read/get) karne ke liye use hota hai.

Syntax

```
def get_variable(self):  
    return self.__variable
```

Why Getter is Used?

- Data ko safely access karne ke liye
- Encapsulation maintain karne ke liye
- Direct access ko avoid karne ke liye

Syntax

```
class Student:
```

```
    def __init__(self):  
        self.__name = "Suraj" # __name private variable hai.
```

```
    def get_name(self):#Getter method banaya.  
        return self.__name # Private variable ki value return kar raha hai.
```

```
obj = Student()
```

```
print(obj.get_name()) # Getter method se value access ki.
```

What is Setter Method ?

- A Setter Method is used to set/update/change the value of a private variable.
- Setter Method ka use private variable ki value ko set/update/change karne ke liye hota hai.

Why Setter is Used?

- Private data update karne ke liye
- Encapsulation maintain karne ke liye
- Data security improve karne ke liye
- Syntax

```
class Student:
```

```
    def __init__(self):  
        self.__name = "" # Private variable create kiya.
```

```
    def set_name(self, name): # Setter method banaya.  
        self.__name = name
```

```
    def get_name(self):  
        return self.__name # Private variable ki value update ki.
```

```
obj = Student()
```

```
obj.set_name("Suraj") # Setter method ko call kiya.
```

```
print(obj.get_name())
```

Encapsulation Example

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.__salary = salary
    def get_salary(self):
        return self.__salary
    def set_salary(self, amount):
        if amount > 0:
            self.__salary = amount
    def show(self):
        print("Name :", self.name)
        print("Salary :", self.__salary)
e1 = Employee("Suraj", 50000)
e1.show()
e1.set_salary(80000)
print("\nUpdated Data")
e1.show()
```

Data Validation

Data Validation is the process of checking whether the input data is correct and valid before

Data Validation ka matlab hai object me data store karne se pehle check karna ki data sahi (valid) hai ya nahi. Agar data galat ho to use accept nahi kiya jata.

Why Data Validation is Used?

- Prevents invalid data
- Improves data security
- Maintains data accuracy
- Protects object data

Syntax

```
class ClassName:  
    def __init__(self):  
        self.__variable = value  
  
    def set_variable(self, value):  
        if condition:  
            self.__variable = value  
        else:  
            print("Invalid Data")  
    def get_variable(self):  
        return self.__variable
```


Age Validation

```
class Student:
    def __init__(self):
        self.__age = 18

    def set_age(self, age):
        if age >= 18:
            self.__age = age
        else:
            print("Invalid Age")

    def get_age(self):
        return self.__age

s = Student()
s.set_age(20)
print(s.get_age())
s.set_age(15)
print(s.get_age())
```

Explanation:

- 20 valid hai, isliye age update ho gayi.
- 15 valid nahi hai (condition age >= 18 fail), isliye age change nahi hui.

Salary Validation

```
class Employee:
    def __init__(self):
        self.__salary = 10000
    def set_salary(self, salary):
        if salary > 0:
            self.__salary = salary
        else:
            print("Invalid Salary")
    def get_salary(self):
        return self.__salary

e = Employee()
e.set_salary(50000)
print(e.get_salary())
e.set_salary(-1000)
print(e.get_salary())
```

Marks Validation

```
class Student:
    def __init__(self):
        self.__marks = 0

    def set_marks(self, marks):
        if 0 <= marks <= 100:
            self.__marks = marks
        else:
            print("Marks should be between 0 and 100")

    def get_marks(self):
        return self.__marks

s = Student()
s.set_marks(85)
print(s.get_marks())
s.set_marks(120)
print(s.get_marks())
```

Polymorphism

- **Polymorphism** means "many forms." It allows the same method or function name to perform different tasks depending on the object or situation.
- **Polymorphism** ka matlab "ek hi naam, alag-alag kaam" hota hai. Yaani ek hi method alag objects ke liye alag behavior dikhata hai.

Types of Polymorphism

Python me mainly **2 types** ke Polymorphism hote hain.

1. Compile-Time Polymorphism (Method Overloading)

- **Method Overloading** means creating multiple methods with the same name but different parameters.
- Ek hi naam ke method ko alag-alag parameters ke saath banana.
- **Note:** Python me traditional Method Overloading support nahi hota. Agar same naam ka method baar-baar likhenge, to last wala method pehle wale ko replace kar deta hai.

Syntax

```
class ClassName:
    def method_name(self, parameter1):
        # Code
    def method_name(self, parameter1, parameter2):
        # Code
    def method_name(self, parameter1, parameter2, parameter3):
        # Code
```

Example (Traditional Overloading Concept)

```
class Calculator:
    def add(self, a, b):
        print(a + b)
    def add(self, a, b, c):
        print(a + b + c)

obj = Calculator()
obj.add(10, 20, 30)
```

Explanation

- Pehla add () method replace ho gaya.
- Sirf last wala add (a, b, c) method exist karta hai.

Example(Python Way using Default Arguments)

- Python me Method Overloading ki jagah **default arguments** use kiye jaate hain.

```
class Calculator:  
    def add(self, a, b, c=0):  
        print(a + b + c)
```

```
obj = Calculator()  
obj.add(10, 20)  
obj.add(10, 20, 30)
```

Example (Python Way using *args)

```
class Calculator:  
    def add(self, *numbers):  
        print(sum(numbers))
```

```
obj = Calculator()  
obj.add(10, 20)  
obj.add(10, 20, 30)  
obj.add(10, 20, 30, 40)
```

Interview Points

- **Compile-Time Polymorphism = Method Overloading**
- **Python traditional Method Overloading support nahi karta.**
- Python me Method Overloading ko achieve karne ke liye:
- Default Arguments (c=0)
- Variable-Length Arguments (*args)
ka use kiya jata hai.

2. Runtime Polymorphism (Method Overriding)

- Method Overriding means a child class provides its own implementation of a method already defined in the parent class.
- Child class, Parent class ke method ko apne hisaab se dobara likhti hai.

Syntax (Method Overriding)

```
class Parent:
    def method(self):
        print("Parent Method")

class Child(Parent):
    def method(self):
        print("Child Method")

obj = Child()
obj.method()
```

Example

```
class Animal:
    def sound(self):
        print("Animal makes sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

d = Dog()
d.sound()
```

Explanation

- Animal class me **sound()** method hai.
- Dog class ne usi **sound()** method ko override kiya.
- Jab Dog ka object bana aur **sound()** call hua, to **child class ka method** execute hua.

Abstraction

- Abstraction is the process of hiding the implementation details and showing only the essential features to the user.
- The user knows what an object does, but not how it does it.
- Abstraction ka arth hai implementation (ander ki karyapranaali/logic) ko chhupana aur user ko keval zaruri features (functionality) dikhana. isme user ko keval yah pata hota hai ki object kya karta hai (What), lekin wah kaise karta hai (How) yah chhupa rahata hai.

Why Use Abstraction?

- Hide implementation details
- Increase security
- Reduce complexity
- Easy to use
- Easy maintenance
- Better code organization

Syntax

```
# Imports the ABC class and abstractmethod decorator.
from abc import ABC, abstractmethod
class Parent(ABC): # Parent becomes an Abstract Class.
    @abstractmethod # Creates an Abstract Method.
    def display(self): # Only declaration is given.
        pass # No implementation.
class Child(Parent): # Child inherits Parent.
    def display(self):
        print("Abstraction Example")# Child provides the implementation.
obj = Child() # Creates an object and calls the method.
obj.display()
```

Example

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass

class Bike(Vehicle):
    def start(self):
        print("Bike Started")

b = Bike()

b.start()
```

Important Point

English	Hindi
Abstract Class	अमूर्त (Abstract) Class
Abstract Method	केवल declaration वाला method
Cannot create object of Abstract Class	Abstract Class का object नहीं बना सकते
Child class must implement abstract methods	Child class को abstract method implement करना जरूरी है
Uses ABC	ABC से abstract class बनती है
Uses @abstractmethod	@abstractmethod से abstract method बनता है

Another Example

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

class Dog(Animal):
    def sound(self):
        print("Bark")

d = Dog()

d.sound()
```


Types of Abstraction

What is Abstract Class

- An Abstract Class is a class that contains one or more abstract methods. It cannot be instantiated, meaning you cannot create an object of an abstract class directly. It is used as a blueprint for other classes.
- Abstract Class ek aisi class hoti hai jisme ek ya ek se adhik abstract methods hote hain. Is class ka object nahi banaya ja sakta. Ye dusri classes ke liye blueprint (base class) ka kaam karti hai.

Syntax

```
from abc import ABC, abstractmethod  
class Shape(ABC):  
  
    @abstractmethod  
    def area(self):  
        pass
```

```
from abc import ABC, abstractmethod
```

```
# Abstract Class
```

```
class Shape(ABC):  
    @abstractmethod  
    def area(self):  
        pass
```

```
# Child Class
```

```
class Rectangle(Shape):  
    def __init__(self, length, width):  
        self.length = length  
        self.width = width
```

```
# Object of Child Class
```

```
r1 = Rectangle(10, 5)  
print("Area =", r1.area())
```

What Happens if We Create an Object of Abstract Class?

```
from abc import ABC, abstractmethod  
class Shape(ABC):  
    @abstractmethod  
    def area(self):  
        pass  
  
# Error  
s1 = Shape()
```

Explanation (Step by Step)

- **ABC** → Shape ko **Abstract Class** banata hai.
- **@abstractmethod** → area() ko **Abstract Method** banata hai.
- **pass** → Sirf method declare ki gayi hai, implementation nahi di gayi.
- Rectangle class, Shape ko inherit karti hai.
- Rectangle me area() method ki implementation di gayi.
- Ab Rectangle ka object ban sakta hai.
- Lekin Shape ka object **kabhi nahi ban sakta**.

Interface (Using Abstract Class)

What is Interface (Using Abstract Class)

- An Interface is a structure that defines what a class should do, but not how it should do it.
- In Python, there is no separate `interface` keyword. We create an interface using an Abstract Class with abstract methods.
- Interface ek aisa structure hai jo batata hai ki class ko kya kaam karna hai, lekin kaise karna hai ye child class decide karti hai.
- Python me Java ki tarah alag `interface` keyword nahi hota. Python me Abstract Class aur `@abstractmethod` ka use karke Interface banaya jata hai.

Syntax

```
from abc import ABC, abstractmethod  
class Animal(ABC):  
    @abstractmethod  
    def sound(self):  
        pass
```

Example

```
from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

class Dog(Animal):
    def sound(self):
        print("Dog Barks")

d = Dog()
d.sound()
```

Example 2 (Multiple Methods)

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass

    @abstractmethod
    def stop(self):
        pass

class Car(Vehicle):
    def start(self):
        print("Car Started")

    def stop(self):
        print("Car Stopped")

c = Car()
c.start()
c.stop()
```